

IUT Nantes
Pôle Sciences et technologie

Département Informatique - BUT 2

Ressource R4.03 : Qualité et au delà du relationnel

Gilles Nachouki

Le plan i

Chapitre 1 - Le modèle relationnel et qualité du schéma

Concepts de base

Domaine

Dépendance fonctionnelle

Fermeture d'un ensemble de DFs

Couverture minimale d' un ensemble de DFs

Algorithme de recherche d"une couverture minimale

Exemple

Normalisation

Pourquoi normaliser ?

Quand normaliser ?

1 FN : attributs simples

2 FN : DFs élémentaires à partir de la clé

Le plan ii

3 FN : DFs élémentaires et directes à partir de la clé

3 FNBC : les seules sources de dépendances sont des clés de R

Qualité de schéma

Comment normaliser ou décomposer une relation

Algorithmes

Chapitre 2 - Au delà du modèle relationnel

NOSQL et les différents modèles de données

Modélisation orientée documents

Le système MongoDB

Principales commandes MongoDB et CRUD

Chapitre 1 - Le modèle relationnel et qualité du schéma

Le modèle relationnel (rappel)

- Créé en 1970 par E.F Codd
- Fondé sur la théorie des ensembles et sur le concept de relation

Concepts de base

Domaine :

Un domaine est un ensemble de valeurs (au sens théorie des ensembles)sur lequel une fonction ou une relation peut être définie.

Domaine :

Un domaine est un ensemble de valeurs (au sens théorie des ensembles)sur lequel une fonction ou une relation peut être définie.

Exemple
le domaine des noms, le domaine des âges,...
le domaine des entiers E={...-1,0,+1,+2,...}

Attribut :

Attribut :
Un attribut A_i est une variable qui prend ses valeurs dans un ensemble D_i appelé domaine.

Exemple
noEtudiant :entier ; nomEtudiant :nom, âgeEtudiant :âge,...

Relation

Soient alors $D_1, D_2, \cdots, D_n$, une suite ordonnée de n domaines non forcément distincts, une relation R sur ces n domaines est une partie du produit cartésien $D_1 \times D_2 \times \cdots \times D_n$.

Relation

Soient alors $D_1, D_2, \cdots, D_n$, une suite ordonnée de n domaines non forcément distincts, une relation R sur ces n domaines est une partie du produit cartésien $D_1 \times D_2 \times \cdots \times D_n$.

Les éléments retenus dans le produit sont ceux qui correspondent à des informations dans le réel considéré.

Relation

Soient alors $D_1, D_2, \cdots, D_n$, une suite ordonnée de n domaines non forcément distincts, une relation R sur ces n domaines est une partie du produit cartésien $D_1 \times D_2 \times \cdots \times D_n$.

Les éléments retenus dans le produit sont ceux qui correspondent à des informations dans le réel considéré.

Une relation est un ensemble de tuples $(d_1, d_2, \cdots, d_n)$ tels que :

$$d_1 \in D_1, d_2 \in D_2 \cdots, d_n \in D_n$$

Exemple

noEtudiant :entier	nomEtudiant :nom	âgeEtudiant :âge
3	Robert	45

Relation

Soient alors $D_1, D_2, \cdots, D_n$, une suite ordonnée de n domaines non forcément distincts, une relation R sur ces n domaines est une partie du produit cartésien $D_1 \times D_2 \times \cdots \times D_n$.

Les éléments retenus dans le produit sont ceux qui correspondent à des informations dans le réel considéré.

Une relation est un ensemble de tuples $(d_1, d_2, \cdots, d_n)$ tels que :

$$d_1 \in D_1, d_2 \in D_2 \cdots, d_n \in D_n$$

Exemple

noEtudiant :entier	nomEtudiant :nom	âgeEtudiant :âge
3	Robert	45
5	Raymonde	37

Relation i

N-uplet d' une relation :
Assignation de valeurs à chacun des attributs de la relation. Chaque tuple d' une relation est désigné de façon unique par une valeur de clé.

Exemple

noEtudiant :entier	nomEtudiant :nom	âgeEtudiant :âge
3	Robert	45

Relation ii

Exemple

noEtudiant :entier	nomEtudiant :nom	âgeEtudiant :âge
3	Robert	45
5	Raymonde	37

degré =___ ; cardinalité = ___ ;

Extension d' une relation :
L' extension d' une relation définit un état particulier de la relation à un instant donné.

Relation iii

Exemple

noEtudiant :entier	nomEtudiant :nom	âgeEtudiant :âge
3	Robert	45
5	Raymonde	37

A retenir

- Unicité des noms des attributs
- Indifférence de l'ordre des lignes et des colonnes

Relation i

Schéma de relation :
Le schéma d' une relation est une définition en intention d' une relation.Il contient :

- la liste de ses attributs et leur domaine,
- l' ensemble des contraintes d' intégrité associées à la relation

Ou, plus simplement
doublet constitué de l' ensemble U des attributs et de l' ensemble F des contraintes d' intégrité

$$\boxed{R = <U,F>}$$

Relation ii

Exemple
ETUDIANT <U,F> U={noEtudiant :entier, nomEtudiant :nom,
âgeEtudiant :âge}
F= ensemble de contraintes d'intégrité fonctionnelles

Contrainte d' intégrité fonctionnelle
prédicat permettant de vérifier la validité des opérations de MAJ d' une relation.

Relation normalisée :
Une relation est dite normalisée si ses attributs sont simples (une seule valeur pour chaque tuple)

Exemple

Relation ETUDIANT :

noEtudiant :entier	nomEtudiant	âgeEtudi	adresseEtudiant
3	Robert	45	14,rue du Maines, 44000 Nantes
5	Raymonde	37	12 rue des Roses 44000 Nantes

Solution ?

Relation ETUDIANT :

noEtudiant :entier	nomEtudiant	âgeEtudi	prénomEtudiant
3	Bidochon	45	Robert, Léon
5	Bidochon	37	Raymonde, Rose

Solution ?

Relation : Clé primaire / clé candidate i

On admet généralement que les attributs et les tuples sont tous distincts. Il existe alors au moins un sous-ensemble minimal d' attributs de la relation dont les valeurs permettent de distinguer les tuples les uns des autres. Ce sous ensemble constitue une clé candidate de la relation.

- La valeur d'une clé candidate est donc distincte pour toutes les occurrences.
- Toute relation a au moins une clé candidate et peut en avoir plusieurs. Il ne peut donc jamais y avoir deux occurrences identiques au sein d'une relation.
- Les clés candidates d'une relation n'ont pas forcément le même nombre d'attributs.
- Une clé candidate peut être formée d'un attribut arbitraire, utilisé à cette seule fin.
- Lorsqu' il existe plusieurs clés candidates, l' une d' elles est choisie comme clé primaire.

Une clé étrangère d'une relation X est formée d'un ou plusieurs de ses attributs qui constituent une clé candidate dans une autre relation Y. La relation X est dite référençante et la relation Y est dite référencée.

Exemple i

Etudiant (no-étudiant, nom, prénom,noInsee, adresse)

Matière(no-matière, libellé)

Note(no-étudiant, no-matière, note)

Exemples de clés

Dépendance fonctionnelle i

Définition :
Soit la relation $R(A, B, C)$, l' attribut B est dit fonctionnellement dépendant de l' attribut A si étant donné 2 n-uplets : <a1, b1, c1> et <a2,b2,c2>

$$a1 = a2 \Rightarrow b1 = b2$$

On la note

$$A \rightarrow B$$

A : source de la DF, B : cible de la DF

Exemple :
no-Etudiant→ nomEtudiant
La source peut être un ensemble d' attributs :
(no-Etudiant,no-Matière)→ note

X → Y ou X détermine Y ou Y dépend de X

Dépendance fonctionnelle

Propriétés : Axiomes d'Armstrong
Soit une relation R (A,B,C,G)

- réflexivité : $A \rightarrow A$ ($\forall A$)

Dépendance fonctionnelle

Propriétés : Axiomes d'Armstrong
Soit une relation R (A,B,C,G)

- réflexivité : $A \rightarrow A$ ($\forall A$)
- augmentation : si $A \rightarrow B \Rightarrow A, C \rightarrow B \ \forall C$

Dépendance fonctionnelle

Propriétés : Axiomes d'Armstrong
Soit une relation R (A,B,C,G)

- réflexivité : $A \rightarrow A$ ($\forall A$)
- augmentation : si $A \rightarrow B \Rightarrow A, C \rightarrow B \ \forall C$
- transitivité : si $A \rightarrow B$ et $B \rightarrow C \Rightarrow A \rightarrow C$

Dépendance fonctionnelle

Propriétés : Axiomes d'Armstrong
Soit une relation R (A,B,C,G)

- réflexivité : $A \rightarrow A$ ($\forall A$)
- augmentation : si $A \rightarrow B \Rightarrow A, C \rightarrow B \ \forall C$
- transitivité : si $A \rightarrow B$ et $B \rightarrow C \Rightarrow A \rightarrow C$
- pseudo-transitivité : si $A \rightarrow B$ et $B, C \rightarrow G \Rightarrow A, C \rightarrow G$

Dépendance fonctionnelle

Propriétés : Axiomes d'Armstrong
Soit une relation R (A,B,C,G)

- réflexivité : $A \rightarrow A$ ($\forall A$)
- augmentation : si $A \rightarrow B \Rightarrow A, C \rightarrow B \ \forall C$
- transitivité : si $A \rightarrow B$ et $B \rightarrow C \Rightarrow A \rightarrow C$
- pseudo-transitivité : si $A \rightarrow B$ et $B, C \rightarrow G \Rightarrow A, C \rightarrow G$
- union : si $A \rightarrow B$ et $A \rightarrow C \Rightarrow A \rightarrow B, C$

Dépendance fonctionnelle

Propriétés : Axiomes d'Armstrong
Soit une relation R (A,B,C,G)

- réflexivité : $A \rightarrow A$ ($\forall A$)
- augmentation : si $A \rightarrow B \Rightarrow A, C \rightarrow B \ \forall C$
- transitivité : si $A \rightarrow B$ et $B \rightarrow C \Rightarrow A \rightarrow C$
- pseudo-transitivité : si $A \rightarrow B$ et $B, C \rightarrow G \Rightarrow A, C \rightarrow G$
- union : si $A \rightarrow B$ et $A \rightarrow C \Rightarrow A \rightarrow B, C$
- décomposition : si $A \rightarrow B, C \Rightarrow A \rightarrow B$ et $A \rightarrow C$

Exemple : Axiomes d'Armstrong
Soit la relation R<U,F>

avec

$$U = \{A, B, C\}$$

$$F = \{A \rightarrow B, C; B \rightarrow C\}$$

Exercice : Déduire de F toutes les DFs pouvant être générées en appliquant les propriétés des DFs

$A \rightarrow A, B, C, AB, AC, BC, ABC$
$B \rightarrow B, C, BC$
$C \rightarrow C$
$AB \rightarrow A, B, C, AB, AC, BC, ABC$
$AC \rightarrow A, B, C, AB, AC, BC, ABC$
$BC \rightarrow B, C, BC$
$ABC \rightarrow A, B, C, AB, AC, BC, ABC$

Graphe de dépendances

On peut aussi représenter les Dépendance fonctionnelles sous la forme d'un graphe orienté

Exemple : Graphe de dépendances

$$F = \{A \to B, C; B \to D; AE \to F\}$$

Construire le graphe de dépendances correspondant

Principaux intérêts du graphe de dépendance :

- rechercher une couverture minimale ;
- rechercher une clé de la relation

DF simple :

DF simple :
La DF X $\rightarrow$ Y est dite simple si sa partie droite ne comporte qu'un seul attribut.

Exemple : dépendance simple

DFs

1. no-étudiant $\rightarrow$ nom, prénom
2. idpromotion $\rightarrow$ libelléPromotion

Simple/non-simple

1.
2.

Dépendance élémentaire :

Définition :
La dépendance $X \rightarrow Y$ est dite élémentaire si $\nexists X' \subset X / X' \rightarrow Y$

Dépendance élémentaire :

Définition :
La dépendance $X \rightarrow Y$ est dite élémentaire si $\nexists X' \subset X / X' \rightarrow Y$

Exemple : dépendance élémentaire

DFs

1. $no-étudiant, no-matière \rightarrow note$
2. $no-étudiant, no-matière \rightarrow nom - étudiant$

Elémentaire/non-élémentaire

1.
2.

Dépendance directe :

Définition

La dépendance $X \rightarrow Y$ est dite directe si $\nexists$ un attribut Z de $R / X \rightarrow Z \wedge Z \rightarrow Y$

Dépendance directe :

Définition
La dépendance $X \rightarrow Y$ est dite directe si $\nexists$ un attribut Z de $R / X \rightarrow Z \wedge Z \rightarrow Y$

Exemple : dépendance directe

DFs

1. $no - étudiant, \rightarrow idpromotion$
2. $idpromotion \rightarrow libellé - promotion$
3. $no - étudiant \rightarrow libellé - promotion$

Directe/non directe

1.
2.
3.

Fermeture d 'un ensemble de DFs

Fermeture F^+ de F

Ensemble stationnaire de dépendances obtenu à partir de l' ensemble F initial en appliquant les propriétés des DFs

Exemple :

Déterminer la fermeture F^+ de F :

F=$\{ A \rightarrow B, C ; B \rightarrow C\}$

Fermeture F^+ de F

$\{$

$\{ A \rightarrow A, B, C, AB, AC, BC, ABC$
$B \rightarrow B, C, BC$
$C \rightarrow C$
$AB \rightarrow A, B, C, AB, AC, BC, ABC$
$AC \rightarrow A, B, C, AB, AC, BC, ABC$
$BC \rightarrow B, C, BC$
$ABC \rightarrow A, B, C, AB, AC, BC, ABC \} \}$

Couverture minimale d' un ensemble de DFs

Couverture minimale F_{min} de F :
ensemble minimal de dépendances de F permettant de générer toutes les dépendances de F^+.

Couverture minimale d' un ensemble de DFs

Couverture minimale F_{min} de F :
ensemble minimal de dépendances de F permettant de générer toutes les dépendances de F^+.

$$F^+_{min} = F^+$$

Couverture minimale d' un ensemble de DFs

Couverture minimale F_{min} de F :
ensemble minimal de dépendances de F permettant de générer toutes les dépendances de F^+.

$$F^+_{min} = F^+$$

$$\nexists F' / F' \subset F_{min} \wedge F'^+ = F^+_{min}$$

Remarque : Une relation peut avoir plusieurs couvertures minimales. Elles sont alors équivalentes.

Exemple 1 : couverture minimale

ensemble F de DFs

$$F = \{A \rightarrow B, C; B \rightarrow C\}$$

Exemple 1 : couverture minimale

ensemble F de DFs

$$F = \{A \rightarrow B, C; B \rightarrow C\}$$

Couverture minimale F_{min} de F

$$F_{min} = \{A \rightarrow B; B \rightarrow C\}$$

Exemple 2 : couverture minimale

ensemble F de DFs

$$F = \{A \to B, C; B \to C; C \to B\}$$

Exemple 2 : couverture minimale

ensemble F de DFs

$$F = \{A \rightarrow B, C; B \rightarrow C; C \rightarrow B\}$$

Couverture minimale F_{min} de F

$$F_{min1} = ?$$

$$F_{min2} = ?$$

Calcul de la fermeture d'un ensemble d'attributs dans un ensemble de dépendances

1: $X^+ \leftarrow X$
2: **répéter**
3: $Aux \leftarrow X^+$
4: **pour** chaque DF $Y \dashrightarrow Z$de F, **faire**
5: **si** $Y \subset X^+$ **alors**
6: $X^+ \leftarrow X^+ \cup Z$
7: **finsi**
8: **fin pour**
9: **jusqu'à** Aux $= X^+$ ou $X^+ = R$

Algorithm 1: Calcul de la fermeture d'un ensemble d'attributs dans un ensemble de dépendances

Exemple i

Soit la relation R $< U, F >$ / U={A,B,C,D,E,F,G,H,I,J,K}

F={ $A \rightarrow B, C, E$

$C \rightarrow D, F$

$A, G, B \rightarrow H$

$I \rightarrow J$

$D \rightarrow E$

$E \rightarrow I, J$

}

Déterminer la fermeture de A dans F puis de AG dans F

Algorithme de recherche d' une couverture minimale dans un ensemble de DFs

Définitions préalables

DF simple :

$X \rightarrow A$ où X : attrib. simple ou composé, A : attrib. simple

Algorithme de recherche d' une couverture minimale dans un ensemble de DFs

Définitions préalables

DF simple :

X $\rightarrow$ A où X : attrib. simple ou composé, A : attrib. simple

Attribut superflu :

Soit (X $\rightarrow$ A) $\in$ F, on dit qu' un attribut B est superflu si :

$$(F - \{(X \rightarrow A)\} \cup \{(Z \rightarrow A)\})^+ = F^+ \quad \text{où } Z = X - \{B\}$$

Ou encore

$$A \in Z^+ \text{ où } Z = X - \{B\}$$

Algorithme de recherche d' une couverture minimale dans un ensemble de DFs

Définitions préalables

DF simple :

X $\rightarrow$ A où X : attrib. simple ou composé, A : attrib. simple

Attribut superflu :

Soit (X $\rightarrow$ A) $\in$ F, on dit qu' un attribut B est superflu si :

$$(F - \{(X \rightarrow A)\} \cup \{(Z \rightarrow A)\})^+ = F^+ \quad \text{où } Z = X - \{B\}$$

Ou encore

$$A \in Z^+ \text{ où } Z = X - \{B\}$$

DF redondante :

X $\rightarrow$ A est redondante ssi

$$(F - \{(X \rightarrow A)\})^+ = F^+$$

Ou encore

$$A \in X^+_{(F - \{(X \rightarrow A)\})}$$

Algorithme de recherche d ' une couverture minimale dans un ensemble de DFs

Les étapes de l'algorithme

1. Transformer les DFs en DFs simples ($X \rightarrow Y$)
2. Eliminer les attributs superflus ($X \rightarrow Y$)
3. Eliminer les DFs redondantes ($X \rightarrow Y$)

Exemple i

Soit la relation R $< U, F >$ / U={A,B,C,D,E,F,G,H,I,J,K}

F={ $A \rightarrow B, C, E$

$C \rightarrow D, F$

$A, G, B \rightarrow C, H$

$I \rightarrow J$

$D \rightarrow E$

$E \rightarrow I, J$

}

Déterminer la couverture minimale de R

Exemple :
1) DFs simples i

F_1={
$A \rightarrow B$
$A \rightarrow C$
$A \rightarrow E$
$C \rightarrow D$
$C \rightarrow F$
$A, G, B \rightarrow C$
$A, G, B \rightarrow H$
$I \rightarrow J$
$D \rightarrow E$
$E \rightarrow I$
$E \rightarrow J$
}

Exemple :
2) Attributs superflus

F_1={
$A \rightarrow B$
$A \rightarrow C$
$A \rightarrow E$
$C \rightarrow D$
$C \rightarrow F$
$A, G, B \rightarrow C$
$A, G, B \rightarrow H$
$I \rightarrow J$
$D \rightarrow E$
$E \rightarrow I$
$E \rightarrow J$
}

Exemple :
2) Attributs superflus

$\boxed{A, G, B \rightarrow C}$

- A superflu ? ou encore C $\in GB_F^+$?

Exemple :
2) Attributs superflus

$A, G, B \rightarrow C$

- A superflu ? ou encore C $\in GB_F^+$?
- $GB_0 = \{G, B\}$

Exemple :
2) Attributs superflus

$A, G, B \rightarrow C$

- A superflu ? ou encore C $\in GB_F^+$?
- $GB_0 = \{G, B\}$
- $GB_1 = \{G, B\} = GB_0$

Exemple :
2) Attributs superflus

$A, G, B \rightarrow C$

- A superflu ? ou encore C $\in GB_F^+$?
- $GB_0 = \{G, B\}$
- $GB_1 = \{G, B\} = GB_0$
- Donc C $\notin GB_F^+ \Longrightarrow$ A non superflu

Exemple :
2) Attributs superflus

- G superflu ? ou encore C $\in AB_F^+$?

Exemple :
2) Attributs superflus

- G superflu ? ou encore C $\in AB_F^+$?
- $AB_0 = \{A, B\}$

Exemple :
2) Attributs superflus

- G superflu ? ou encore C $\in AB_F^+$?
- $AB_0 = \{A, B\}$
- $AB_1 = \{A, B, C, E\}$

Exemple :
2) Attributs superflus

- G superflu ? ou encore C $\in AB_F^+$?
- $AB_0 = \{A, B\}$
- $AB_1 = \{A, B, C, E\}$
- $AB_2 = \{A, B, C, E, D, F, I, J\}$

Exemple :
2) Attributs superflus

- G superflu ? ou encore C $\in AB_F^+$?
- $AB_0 = \{A, B\}$
- $AB_1 = \{A, B, C, E\}$
- $AB_2 = \{A, B, C, E, D, F, I, J\}$
- $AB_3 = \{A, B, C, E, D, F, I, J\} = AB_2$

Exemple :
2) Attributs superflus

- G superflu ? ou encore C $\in AB_F^+$?
- $AB_0 = \{A, B\}$
- $AB_1 = \{A, B, C, E\}$
- $AB_2 = \{A, B, C, E, D, F, I, J\}$
- $AB_3 = \{A, B, C, E, D, F, I, J\} = AB_2$
- C $\in AB_F^+ \Longrightarrow$ G superflu

Exemple :
2) Attributs superflus

- G superflu ? ou encore C $\in AB_F^+$?
- $AB_0 = \{A, B\}$
- $AB_1 = \{A, B, C, E\}$
- $AB_2 = \{A, B, C, E, D, F, I, J\}$
- $AB_3 = \{A, B, C, E, D, F, I, J\} = AB_2$
- C $\in AB_F^+ \Longrightarrow$ G superflu
- $A, G, B \rightarrow C$ devient $A, B \rightarrow C$

Exemple :
2) Attributs superflus

- B superflu ? ou encore C $\in A_F^+$?

Exemple :
2) Attributs superflus

- B superflu ? ou encore C $\in A_F^+$?
- $A_0 = \{A\}$

Exemple :
2) Attributs superflus

- B superflu ? ou encore C $\in A_F^+$?
- $A_0 = \{A\}$
- $A_1 = \{A, B, C, E\}$

Exemple :
2) Attributs superflus

- B superflu ? ou encore C $\in A_F^+$?
- $A_0 = \{A\}$
- $A_1 = \{A, B, C, E\}$
- $A_2 = \{A, B, C, E, D, F, I, J\}$

Exemple :
2) Attributs superflus

- B superflu ? ou encore C $\in A_F^+$?
- $A_0 = \{A\}$
- $A_1 = \{A, B, C, E\}$
- $A_2 = \{A, B, C, E, D, F, I, J\}$
- $A_3 = \{A, B, C, E, D, F, I, J\} = A_2$

Exemple :
2) Attributs superflus

- B superflu ? ou encore C $\in A_F^+$?
- $A_0 = \{A\}$
- $A_1 = \{A, B, C, E\}$
- $A_2 = \{A, B, C, E, D, F, I, J\}$
- $A_3 = \{A, B, C, E, D, F, I, J\} = A_2$
- C $\in A_F^+ \Longrightarrow$ B superflu

Exemple :
2) Attributs superflus

- B superflu ? ou encore C $\in A_F^+$?
- $A_0 = \{A\}$
- $A_1 = \{A, B, C, E\}$
- $A_2 = \{A, B, C, E, D, F, I, J\}$
- $A_3 = \{A, B, C, E, D, F, I, J\} = A_2$
- C $\in A_F^+ \Longrightarrow$ B superflu
- $A, B \rightarrow C$ devient $A \rightarrow C$

Exemple :
2) Attributs superflus

F_1={
$A \rightarrow B$
$A \rightarrow C$
$A \rightarrow E$
$C \rightarrow D$
$C \rightarrow F$
$(A \rightarrow C)$
$A, G, B \rightarrow H$
$I \rightarrow J$
$D \rightarrow E$
$E \rightarrow I$
$E \rightarrow J$
}

Exemple :
2) Attributs superflus

$\boxed{A, G, B \rightarrow H}$

- A superflu ? ou encore H $\in GB_F^+$?

Exemple :
2) Attributs superflus

$A, G, B \rightarrow H$

- A superflu ? ou encore H $\in GB_F^+$?
- $GB_0 = \{G, B\}$

Exemple :
2) Attributs superflus

$A, G, B \rightarrow H$

- A superflu ? ou encore H $\in GB_F^+$?
- $GB_0 = \{G, B\}$
- $GB_1 = \{G, B\} = GB_0$

Exemple :
2) Attributs superflus

$\boxed{A, G, B \rightarrow H}$

- A superflu ? ou encore H $\in GB_F^+$?
- $GB_0 = \{G, B\}$
- $GB_1 = \{G, B\} = GB_0$
- Donc H $\notin GB_F^+ \Longrightarrow$ A non superflu

Exemple :
2) Attributs superflus

- G superflu ? ou encore H $\in AB_F^+$?
- $AB_F^+ = \{$A,B,C,E,D,F,I,J$\}$
- H $\notin AB_F^+ \Longrightarrow$ G non superflu

Exemple :
2) Attributs superflus

- B superflu ? ou encore H $\in AG_F^+$?
- $AG_0 = \{A, G\}$
- $AG_1 = \{A, B, C, E, G\}$
- $AG_2 = \{A, B, C, E, D, F, G, H, I, J\}$
- $AG_3 = AG_2$
- H $\in AG_F^+ \Longrightarrow$ B superflu
- $A, G, B \rightarrow H$ devient $A, G \rightarrow H$

Exemple :
2) Attributs superflus

$F_2 = \{$
$A \rightarrow B$
$A \rightarrow C$
$A \rightarrow E$
$E \rightarrow J$
$C \rightarrow D$
$C \rightarrow F$
$A, G \rightarrow H$
$I \rightarrow J$
$D \rightarrow E$
$E \rightarrow I$
$E \rightarrow J$
$\}$

Exemple :
3) DFs redondantes

F_2={

$A \rightarrow B$

$A \rightarrow C$

$A \rightarrow E$

$E \rightarrow J$

$C \rightarrow D$

$C \rightarrow F$

$A, G \rightarrow H$

$I \rightarrow J$

$D \rightarrow E$

$E \rightarrow I$

}

Exemple :
3) DFs redondantes

$\boxed{A \to B \text{ redondante ?}}$

$$B \in A^{+}_{F-\{A \to B\}} ?$$

- $A_0 = \{A\}$

Exemple :
3) DFs redondantes

$A \to B$ redondante ?

$$B \in A^{+}_{F-\{A \to B\}} ?$$

- $A_0 = \{A\}$
- $A_1 = \{A, C, E\}$

Exemple :
3) DFs redondantes

$A \to B$ redondante ?

$$B \in A^+_{F-\{A \to B\}} ?$$

- $A_0 = \{A\}$
- $A_1 = \{A, C, E\}$
- $A_2 = \{A, C, E, D, F, I, J\}$

Exemple :
3) DFs redondantes

$A \to B$ redondante ?

$$B \in A^+_{F-\{A \to B\}} ?$$

- $A_0 = \{A\}$
- $A_1 = \{A, C, E\}$
- $A_2 = \{A, C, E, D, F, I, J\}$
- $A_3 = A_2$

Exemple :
3) DFs redondantes

$A \to B$ redondante ?

$$B \in A^+_{F-\{A \to B\}} ?$$

- $A_0 = \{A\}$
- $A_1 = \{A, C, E\}$
- $A_2 = \{A, C, E, D, F, I, J\}$
- $A_3 = A_2$
- Donc $B \notin A^+_{F-\{A \to B\}}$ et $A \to B$ non redondante

Exemple :
3) DFs redondantes

$A \to C$ redondante ?

$$C \in A^+_{F-\{A \to C\}} ?$$

- $A_0 = \{A\}$
- $A_1 = \{A, B, E\}$
- $A_2 = \{A, B, E, I, J\}$
- $A_3 = A_2$
- Donc $C \notin A^+_{F-\{A \to C\}}$ et $A \to C$ non redondante

Exemple : 3) DFs redondantes

- $A_0 = \{A\}$
- $A_1 = \{A, B, C\}$
- $A_2 = \{A, B, C, D, F\}$
- $A_3 = \{A, B, C, D, E, F\}$
- Donc $E \in A^+_{F-\{A \to E\}}$ et $A \to E$ redondante
- *La DF $A \to E$ ne doit plus être prise en compte*

Exemple :
3) DFs redondantes

$E \to J$ redondante ?

$$J \in E^+_{F-\{E\to J, A\to E\}} ?$$

- $E_0 = \{E\}$
- $E_1 = \{E, I\}$
- $E_2 = \{E, I, J\}$
- Donc $J \in E^+_{F-\{E\to J, A\to E\}}$ et $E \to J$ redondante
- La DF $E \to J$ ne doit plus être prise en compte

Exemple :
3) DFs redondantes

$\boxed{C \to D \text{ redondante ?}}$

$$D \in C^{+}_{F-\{C \to D, E \to J, A \to E\}} ?$$

- $C_0 = \{C\}$
- $C_1 = \{C, F\}$
- ...

Mise en oeuvre

Soit l'ensemble de DFs suivant :

$$F = \{A \rightarrow B, C; B \rightarrow C; C \rightarrow B; AC \rightarrow E; EF \rightarrow G\}$$

Déterminer la couverture minimale de F en appliquant l'algorithme précédent.

Solution

$$F_{min1} = \{A \rightarrow B, E; B \rightarrow C; C \rightarrow B; EF \rightarrow G\}$$

$$F_{min2} = \{A \rightarrow C, E; B \rightarrow C; C \rightarrow B; EF \rightarrow G\}$$

Clé d' une relation

Un constituant K est dit clé pour le schéma relationnel <U,F> si :

$$\{K \rightarrow U\} \in F^+$$

$$\nexists K' \subset K / \{K' \rightarrow U\} \in F^+$$

Exemple i

Soit la relation R $< U, F >$ / U={A,B,C,D,E,F,G,H,I,J,K}
F={ $A \rightarrow B, C$
$C \rightarrow D, F$
$A, G, \rightarrow H$
$I \rightarrow J$
$D \rightarrow E$
$E \rightarrow I$
}
Quelle est la clé de la relation ? AGK ?

- Unicité :

$$AGK_F^+ = \{A, B, C, D, E, F, G, H, I, J, K\} = U$$

Exemple ii

- Minimalité :
  $AG_F^+ = \{A, B, C, D, E, F, G, H, I, J\} <> U$
  $AK_F^+ = \{A, B, C, D, E, F, I, J\} <> U$
  $GK_F^+ = \{G, K\} <> U$

Exemple i

Soit la relation R $< U, F >$

U={A,B,C,D}

F={ $A \rightarrow B$; $B \rightarrow A$; $AC \rightarrow D$}

Quelle est la clé de la relation ?

Normalisation

Pourquoi normaliser : Anomalies de MAJ

Exemple

solution 1 :

Relation ETUDIANT :

noEtudiant	nomEtudiant	codePromo	libelléPromo
3	Robert	INFO	Informatique
5	Raymonde	INFO	Informatique
7	Kador	ETN	Electronique

Pourquoi normaliser : Anomalies de MAJ

Exemple

solution 2 :

Relation ELEVE :

noEtudiant	nomEtudiant	codePromo
3	Robert	INFO
5	Raymonde	INFO
7	Kador	ETN

Relation PROMOTION :

codePromo	libelléPromo
INFO	Informatique
ETN	Electronique

Pourquoi normaliser : Anomalies de MAJ

Exemple

solution 1 :

Relation ETUDIANT :

noEtudiant	nomEtudiant	codePromo	libelléPromo
3	Robert	INFO	Informatique
5	Raymonde	INFO	Informatique
7	Kador	ETN	Electronique

ajout d'une nouvelle promo : GE

Pourquoi normaliser : Anomalies de MAJ

Exemple

solution 1 :

Relation ETUDIANT :

noEtudiant	nomEtudiant	codePromo	libelléPromo
3	Robert	INFO	Informatique
5	Raymonde	INFO	Informatique
7	Kador	ETN	Electronique

modification du libellé de la promo Info : Cybersécurité

+Pourquoi normaliser : Anomalies de MAJ

Exemple

solution 1 :

Relation ETUDIANT :

noEtudiant	nomEtudiant	codePromo	libelléPromo
3	Robert	INFO	Informatique
5	Raymonde	INFO	Informatique
7	Kador	ETN	Electronique

suppression de l'étudiant Kador

Pourquoi normaliser : Anomalies de MAJ

solution 2 :

Relation ELEVE :

noEtudiant	nomEtudiant	codePromo
3	Robert	INFO
5	Raymonde	INFO
7	Kador	ETN

Relation PROMOTION :

codePromo	libelléPromo
INFO	Informatique
ETN	Electronique

insertion ajout d'une nouvelle promo : GE

Pourquoi normaliser : Anomalies de MAJ

solution 2 :

Relation ELEVE :

noEtudiant	nomEtudiant	codePromo
3	Robert	INFO
5	Raymonde	INFO
7	Kador	ETN

Relation PROMOTION :

codePromo	libelléPromo
INFO	Informatique
ETN	Electronique

insertion ajout d'une nouvelle promo : GE

modification modification du libellé de la promo Info : Cybersécurité

Pourquoi normaliser : Anomalies de MAJ

solution 2 :

Relation ELEVE :

noEtudiant	nomEtudiant	codePromo
3	Robert	INFO
5	Raymonde	INFO
7	Kador	ETN

Relation PROMOTION :

codePromo	libelléPromo
INFO	Informatique
ETN	Electronique

insertion ajout d'une nouvelle promo : GE

modification modification du libellé de la promo Info : Cybersécurité

suppression suppression de l'étudiant Kador

1FN : attributs simples

1FN
Une relation est dite en 1FN si ses attributs sont atomiques (simples et mono-valués). Un attribut est dit atomique s'il ne contient qu'une seule valeur pour un tuple donné, et donc s'il ne regroupe pas un ensemble de plusieurs valeurs.

Exemple
Personne(nom, prénoms, date-naissance, adresse)

2 FN

No-étudiant	Nom	No-matière	Note
5	Bidochon Robert	BD	10
7	Bidochon Raymonde	BD	14
7	Bidochon Raymonde	Info	14
9	Bidochon Kador	Info	18

2 FN

No-étudiant	Nom	No-matière	Note
5	Bidochon Robert	BD	10
7	Bidochon Raymonde	BD	14
7	Bidochon Raymonde	Info	14
9	Bidochon Kador	Info	18

2FN

Une relation est dite en 2FN si tous les constituants non clé sont totalement dépendants des clés.

3 FN

No-étudiant	Nom	Promo	Libellé-promo
5	Robert	ID	Info décisionnelle
7	Raymonde	ID	Info décisionnelle
9	Kador	SILR	Info et Réseaux

3 FN

No-étudiant	Nom	Promo	Libellé-promo
5	Robert	ID	Info décisionnelle
7	Raymonde	ID	Info décisionnelle
9	Kador	SILR	Info et Réseaux

3FN

Une relation est dite en 3FN si tous ses constituants non clés sont directement et totalement dépendants des clés.

3FNBC

Une relation est en 3NFBC (Forme Normale de Boyce Codd) si elle est en troisième forme normale et si les seules sources de dépendances sont des clés de R

Exemple : 3 FNBC

On considère la relation R suivante :

A	B	C
1	2	4
2	2	4
3	2	4
3	5	6
5	5	7

- L'ensemble DF $= \{$ AB $\rightarrow$ C, C $\rightarrow$ B $\}$

Exemple : 3 FNBC

On considère la relation R suivante :

A	B	C
1	2	4
2	2	4
3	2	4
3	5	6
5	5	7

- L'ensemble DF $= \{$ AB $\rightarrow$ C, C $\rightarrow$ B $\}$
- La clé primaire primaire de la relation R est AB. AC est une clé candidate.

Exemple : 3 FNBC

On considère la relation R suivante :

A	B	C
1	2	4
2	2	4
3	2	4
3	5	6
5	5	7

- L'ensemble DF $= \{$ AB $\rightarrow$ C, C $\rightarrow$ B $\}$
- La clé primaire primaire de la relation R est AB. AC est une clé candidate.
- Cette relation est en 3NF mais pas en 3NFBC.

Qualité de schéma

Décomposition sans perte

Théorème de Heath :
Soit une relation R<X,F> décomposée en R1 <X1,F1> et R2<X2,F2>

la décomposition d' une relation est dite sans perte de données ssi :

$$X = X1 \cup X2$$

$$X1 \cap X2 \rightarrow (X1 - X2) \vee (X2 - X1)$$

Décomposition sans perte

Théorème de Heath :
Soit une relation R<X,F> décomposée en R1 <X1,F1> et R2<X2,F2>

la décomposition d' une relation est dite sans perte de données ssi :

$$X = X1 \cup X2$$

$$X1 \cap X2 \rightarrow (X1 - X2) \vee (X2 - X1)$$

La décomposition est dite sans perte de DFs si :

$$F^{+} = (F1 \cup F2)^{+}$$

Décomposition sans perte : Exemple i

R<X,F> avec $X = \{A, B, C\}$,$F = \{A \rightarrow B, B \rightarrow C\}$

Décomposé en

- R1<X1,F1> avec $X1 = \{A, B\}$, $F1 = \{A \rightarrow B\}$
- R2<X2,F2> avec $X2 = \{B, C\}$, $F2 = \{B \rightarrow C\}$,

Cette décomposition est-elle sans perte ?

1. Perte de données ?
   - 1.1 perte d'attributs : $X1 \cup X2 =$
   - 1.2 perte de tuples
     - $X1 \cap X2 =$
     - $X1 - X2 =$
     - $X2 - X1 =$

     L'une ou l'autre des deux propositions est-elle vérifiée ?
     - $X1 \cap X2 \rightarrow X1 - X2$
     - $X1 \cap X2 \rightarrow X2 - X1$
2. perte de DFs ? : $(F1 \cup F2) =$

Comment normaliser ou décomposer une relation

Les principales étapes de la normalisation

1. Vérifier que la relation est en 1ère forme normale
2. Rechercher sa couverture minimale
3. Déterminer la ou les clé(s)de la relation
4. Déterminer sa forme normale
5. Si la relation n'est pas en 3FN, la décomposer en relations mieux normalisées

Algorithmes

Algorithme de synthèse

On part d'une relation et d'un ensemble F de dépendances fonctionnelles.
Le principe de cet algorithme est le suivant :

- on remplace F par une couverture minimale F_{min},

Algorithme de synthèse

On part d'une relation et d'un ensemble F de dépendances fonctionnelles.
Le principe de cet algorithme est le suivant :

- on remplace F par une couverture minimale F_{min},
- on regroupe les dfs ayant même membre gauche X ($X \rightarrow A$). Pour chaque membre gauche X, on définit un schéma de relation contenant tous les attributs intervenant dans ces dfs.

Algorithme de synthèse

On part d'une relation et d'un ensemble F de dépendances fonctionnelles.
Le principe de cet algorithme est le suivant :

- on remplace F par une couverture minimale F_{min},
- on regroupe les dfs ayant même membre gauche X ($X \rightarrow A$). Pour chaque membre gauche X, on définit un schéma de relation contenant tous les attributs intervenant dans ces dfs.
- si aucun des schémas définis à l'étape 2 ne contient de clé R, on ajoute un schéma de relation muni d'une clé de R.

Qualité de schéma

- Chaque schéma issu de cette décomposition est en 3NF ;

Qualité de schéma

- Chaque schéma issu de cette décomposition est en 3NF ;
- Cette décomposition est sans perte de données et sans perte de dépendances fonctionnelles.

Exemple : Algorithme de synthèse

Soit R = (A,B,C,D,E) et F = $\{A \rightarrow B,\ A \rightarrow D,\ B \rightarrow C,\ E \rightarrow D\ \}$

- On part de la couverture minimale de F. On chercher la clé de R : AE.

Exemple : Algorithme de synthèse

Soit R = (A,B,C,D,E) et F = $\{A \rightarrow B,\ A \rightarrow D,\ B \rightarrow C,\ E \rightarrow D\ \}$

- On part de la couverture minimale de F. On chercher la clé de R : AE.
- On regroupe les DF qui ont même membre gauche : R1(A,B,D) F1 = $\{\ A \rightarrow B,\ A \rightarrow D\ \}$
  R2(B, C) F2 = $\{\ B \rightarrow C\ \}$
  R3(E, D) F3 = $\{\ E \rightarrow D\ \}$

Exemple : Algorithme de synthèse

Soit R = (A,B,C,D,E) et F = $\{A \rightarrow B,\ A \rightarrow D,\ B \rightarrow C,\ E \rightarrow D\ \}$

- On part de la couverture minimale de F. On chercher la clé de R : AE.
- On regroupe les DF qui ont même membre gauche : R1(A,B,D) F1 = $\{\ A \rightarrow B,\ A \rightarrow D\ \}$
  R2(B, C) F2 = $\{\ B \rightarrow C\ \}$
  R3(E, D) F3 = $\{\ E \rightarrow D\ \}$
- On ajoute R4(A,E) F = $\{\ \}$
  Cette décomposition est SPI (sans perte d'informations) et SPD (sans perte de dépendances).

Algorithme de décomposition

On part d'une relation R et d'un ensemble F de dépendances fonctionnelles. Le principe de cet algorithme est le suivant :

- on remplace F par une couverture minimale F_{min},

Algorithme de décomposition

On part d'une relation R et d'un ensemble F de dépendances fonctionnelles. Le principe de cet algorithme est le suivant :

- on remplace F par une couverture minimale F_{min},
- trouver dans R une dépendance non triviale $X \rightarrow Y$ telle que X ne contienne pas une clé de R

Algorithme de décomposition

On part d'une relation R et d'un ensemble F de dépendances fonctionnelles. Le principe de cet algorithme est le suivant :

- on remplace F par une couverture minimale F_{min},
- trouver dans R une dépendance non triviale $X \rightarrow Y$ telle que X ne contienne pas une clé de R
- remplacer R par R1($X \bigcup Y$, F1) et R2(U - Y,F2) où F1 et F2 sont deux ensembles de dépendances fonctionnelles obtenues à partir de F,

Algorithme de décomposition

On part d'une relation R et d'un ensemble F de dépendances fonctionnelles. Le principe de cet algorithme est le suivant :

- on remplace F par une couverture minimale F_{min},
- trouver dans R une dépendance non triviale X $\rightarrow$ Y telle que X ne contienne pas une clé de R
- remplacer R par R1(X $\bigcup$ Y, F1) et R2(U - Y,F2) où F1 et F2 sont deux ensembles de dépendances fonctionnelles obtenues à partir de F,
- pour tout Ri(Xi,Fi) et Rj(Xj,Fj) tels que Xi $\subseteq$ Xj supprimer Ri.

Qualité de schéma

- Chaque schéma issue de cette décomposition est en 3NFBC ;

Qualité de schéma

- Chaque schéma issue de cette décomposition est en 3NFBC ;
- Cette décomposition est sans perte de données mais elle ne garantie pas de retrouver toutes les dépendances fonctionnelles.

Exemple : Algorithme de décomposition

Soit R = (A,B,C,D,E) et F = $\{A \rightarrow B,\ A \rightarrow D,\ B \rightarrow C,\ E \rightarrow D\ \}$

- On part de la couverture minimale de F. On chercher la clé de R AE.

Exemple : Algorithme de décomposition

Soit R = (A,B,C,D,E) et F = $\{A \rightarrow B, A \rightarrow D, B \rightarrow C, E \rightarrow D\}$

- On part de la couverture minimale de F. On chercher la clé de R AE.
- On décompose selon la DF suivante : $A \rightarrow B$. On obtient :
  R1(A,B), F1 = $\{A \rightarrow B\}$ et R2 (A,C,D,E) et F2 = $\{A \rightarrow D, E \rightarrow D\}$. La clé de R2 est ACE. On décompose selon la DF $A \rightarrow D$. On obtient :
  R21(A,D), F21 = $\{A \rightarrow D\}$ et R22(A,C,E), F22 = $\{\}$.
  Cette décomposition est sans perte de données mais avec perte de DF.

Chapitre 2 - Au delà du modèle relationnel

Introduction

Le modèle de données relationnel et la normalisation des données ont montré leurs limites. En effet, avec l'augmentation du volume de données manipulé (centaines-milliers Teraoctets), le modèle de données non relationnel est apparu et s'est imposé en proposant d'abandonner quelques contraintes du relationnel et de se focaliser sur la manière de stocker et d'interroger ces données.

Plusieurs modèles de données

Le NOSQL (Not Only SQL) est une base de données non relationnelle. Les systèmes de gestion de base de données correspondants permettent le stockage et l'analyse du Big Data (ou données massives). Plusieurs modèles de données sont apparus pour gérer les données dans les bases de données NOSQL :

1. Clés/Valeurs
2. Colonnes
3. Documents
4. Graphes

Le modèle de données qui sera choisi dépendra des besoins de l'application

Modèle de données orienté *Clé/Valeur*

1- Représentation simple des objets

2- La *clé* identifie de façon unique l'objet et *valeur* contient n'importe quel type de données.

3- Cette façon de décrire les données peut être vue comme une table de hachage (distribuée) facilitant ainsi la recherche des données sur le réseau.

Exemples d'implémentation : Rebis, SimpleDB
Applications : e-commerce, réseau social.

Modèle de données orienté *Clé/Valeur*

1- Ce modèle permet la prise en charge de larges volumes de données

2- Des performances élevées en lecture et en écriture

3- Absence de langage de requêtes. Les requêtes sont basées uniquement sur les clés

Modèle de données orienté *Clé/Valeur*

On considère une base de données contenant deux tables Livre et Auteur.
Livre(isbn,titre, ida) et Auteur(ida,nom,adr)

Une modélisation possible orientée Clé/valeur est la suivante :

Clé : ISBN (ex. 1-2735)
Valeur : {titre : 'Base de données', { ida :12, nom :'Martin',
adr :'Nantes'}}

Modèle de données orienté *Colonne*

1- Ce type de stockage s'intéresse aux colonnes au lieu des lignes dans les systèmes classiques. En effet, dans les SGBDR le nombre de colonne pour chaque tuple (ligne) est le même.

2- Les bases de données orientée Colonne permettent d'avoir des colonnes différentes pour chaque tuple.

3- Ce modèle permet d'optimiser le stockage des données associées
- Systèmes : BigTable, Hbase, Cassandra etc.
- Modèle adapté aux applications nécessitant d'accueillir des millions de colonnes dans une table.
- Performances

Modèle de données orienté *Colonne*

On considère la relation R suivante :

id	A	B	C	D	E
1	10	2	3		
2		1			
3			0	2	4

Table 1 – R

Modèle de données orienté *Colonne*

La représentation de R dans une BDD orientée colonnes est comme suit :
a chaque ID de ligne correspond une liste de couples clé/valeurs

id	valeurs
1	(A,10),(B,2),(C,3)
2	(B,1)
3	(C,0),(D,2),(E,4)

Table 2 – R

1. Un stockage optimisé des données
2. L'accès aux données est plus rapide

Modèle de données orienté *Document*

3- Ce modèle repose sur le modèle Clé/Valeur avec une flexibilité accrue car il permet de stocker des valeurs complexes sous forme hierarchique semi-structuré de type JSON. A la différence du modèle Clé/Valeur, une valeur dans le modèle orienté document peut être examinée par une requête.

Systèmes : MongoDB, CouchDB etc.
Applications : il permet d'effectuer des requêtes ; un modèle simple mais puissant ; il permet de représenter des liens de type one-to-one et one-to-many

Modèle de données orienté *Graphe*

3- Dans ce modèle les données sont représentées par des noeuds et les liens entre les noeuds représentent des liens sémantiques entre les données dans les noeuds.

Systèmes : Neo4J, FlockDB etc.
Applications : réseaux (sociaux, électrique etc.)

Caractéristiques des bases de données NOSQL

1. Différents modèles de stockage de données
2. Absence de jointure : les bases de données NOSQL sont capables d'extraire les données sans effectuer de jointure
3. Abasence de schéma : Un utilisateur n'a pas besoin de définir le schéma de la base de données NOSQL qui continue de changer dans le temps
4. Plusieurs serveurs : la plupart des systèmes NOSQL offrent la possibilité de stocker une base de données sur plusieurs serveurs tout en gardant de bonnes performances

Propriétés des bases de données NOSQL

Les propriétés ACID des SGBDR ne sont pas applicables dans un contexte distribué tel que NOSQL (provoquent des latences dans les transactions). Les propriétés suivantes appelées **BASE** ont été proposées dans ce cadre dans le but est de relâcher la pression sur les transactions :

BA *Basically Available :* le système doit garantir un taux de disponibilité des données

S *Soft-state :* la base de données n'a pas à être cohérente à tout instant

E *Eventually consistent :* A terme, la base atteint un état cohérent

Avantages et inconvénients des systèmes relationnels

1. Les contraintes d'intégrités doivent être respectées
2. Les vues assurent la confidentialité des données
3. Les vues simplifient l'écriture des requêtes
4. Les utilisateurs sont familiarisés avec le modèle relationnel
5. Comprendre et définir le schéma de la base de données. La modification du schéma peut être couteuse
6. Les jointures affectent les performances du système
7. Les performances sont limitées pour un très grand volume de données (des Teraoctets)
8. L' évolutivité se fait de manière verticale (cad. la mise à l'échelle se fait en ajoutant davantage de puissance mémoires ou CPU)

Avantages et inconvénients du système non relationnel

1. Distribution des données sur les serveurs
2. l' évolutivité se fait de manière horizontale en ajoutant plus de serveurs
3. Absence de jointures
4. Absence de schéma
5. le paradigme NOSQL est une technologie récente pour beaucoup d'utilisateurs
6. Absence de standard au niveau des langages de requêtes
7. Des modèles de données où chacun répond à un besoin précis

Dans le cadre d'un SGBDR la première étape consiste à concevoir le schéma de la base de données avant de déterminer les requêtes ou les applications qui utiliseront cette base
Dans le contexte NOSQL il est important de connaitre le type d'application qui utilise le futur base avant de commencer l'étape de modélisation.

Pour la modélisation de données nous distinguons trois types d'association entre les entités (repris du modèle Entité-Association) :

1. 1-1
2. 1-N
3. N-N

Les associations entre les entités dans les documents peuvent être exprimés sous les deux formes suivantes :

1. 1-1 : par integration
2. 1-N : par integration
3. N-N : par référence

Lien de type 1-1 :
Nous considérons le cas où une personne peut être mariée (ou pacsée) avec une autre personne.
Pour modéliser cette situation, nous utilisons la première forme par intégration pour représenter cette association. Dans ce cas le lien entre les deux personnes se trouve intégré dans un même document en ajoutant les propriétés respectives des personnes : nom, prénom, age etc.

Lien de type 1-N :
Nous considérons le cas où une personne possède plusieurs voitures.

Pour modéliser cette situation, nous utilisons la forme d'intégration pour représenter cette relation. Dans ce cas, une personne est décrite avec toutes ses voitures sous forme d'un tableau dans le même document avec leurs propriétés respectives.

Lien de type N-N :

Nous considérons le cas où dans un Bon de Commande (BC) il y a plusieurs produits qui sont référencés et qu'un produit peut être référencé dans plusieurs BC.

Pour modéliser cette situation, nous utilisons la deuxième forme de référence pour représenter les liens entre les deux entités BC et Produit : les BC et les produits sont stockés dans des documents différents et chacun fait référence à l'autre (une commande fait référence à ses produit et inversement).

Principaux composants de Mongodb

Mongodb est un système de bases de données non-relationnel orienté document. On distingue principalement les trois composants suivants :

1- Base de données

2- Collection

3- Document

Principaux composants

1- `Base de données` : Il constitue le composant principal de MongoDB. Une base de données Mongo est un conteneur des structures de données appelées collections. Une base de données Mongo gère ses propres fichiers physiques. Un serveur MongoDB héberge plusieurs bases de données

2- `Collection` : Une collection est un ensemble de documents. Par analogie, une collection représente une table dans une base de données relationnelle. Une collection est unique mais plusieurs collections peuvent être présentes en même temps dans une base de données Mongo.
Une collection n'est pas soumise au respect d'un schéma conceptuel de données comme dans le cadre d'une base de données relationnelle.

Principaux composants

3- `Document` : Un document constitue l'unité de base dans une base de données Mongo. Un document est composé d'un ensemble de clés/valeurs. A l'inverse d'une base de données classique les documents dans une collection peuvent contenir des champs différents

Stockage de documents : JSON

MongoDB stocke les documents dans le langage **JSON** (JavaScript Object Notation). JSON est un langage standard utilisé dans des applications pour échanger des données de la même façon que le langage XML.

Au niveau interne MongoDB représente les documents JSON en utilisant un format d'encodage binaire appelé **BSON** (Binary JSON). BSON ajoute à JSON la prise en charge de quelques types de données tels que Date et Binaire qui ne sont pas pris en charge dans JSON.

Types de données

1. String
2. Integer
3. Boolean
4. Double
5. Arrays
6. Object
7. Date

Exemples de document dans MongoDb

Un document : {"_id" :1, "nom" : "Dupond",
"age" :25,
"tél" : 06-45-78-09-12"},
"articles" : ["livre", "stylo"],
Un autre document : {"_id" :2,"nom" : "Marion",
"Age" :35,
"tél" : 07-45-78-23-12",
"adresse" : {
"n°" : "15",
"nom" : "rue du Lila",
"CP" : 75000,
"Ville" : "Paris" }

Chaque document a un identifiant. MongoDB est sensible au type et à la casse (majuscule, minuscule)

Lancement et choix de la base de données

mongo Cette commande permet de lancer Mongo (équivalent à sqlplus dans Oracle). Le client essaie de se connecter au serveur Mongo. Au cas où le nom de la base de données n'est pas spécifié au lancement du client, ce dernier choisit une base de données par défaut appelée **Test**

use ExBd Cette commande pointe, si elle existe, sur la base de données **ExBd**. Dans le cas contraire mongo crée la base de données. *Dans MongoDB une collection est créée lors de l'insertion d'un document*

show dbs Cette commande permet d'afficher la liste des bases de données existantes dans Mongodb. `Les bases de données listées sont celles qui contiennent au moins une collection`

`Insertion de documents dans une collection`

Les documents Mongodb sont spécifiés dans le format JSON.

insert la commande suivante permet d'insérer dans la collection *personnes* un document dans la base de données :
db.personnes.insert({"nom" :"Martin","age" :30})

Consultation de tous les documents d'une collection

find la commande suivante permet de consulter l'ensemble des documents dans **personnes** : db.personnes.find().

La réponse de Mongodb est retournée comme suit :
{"_id" :ObjectId("550744..."),"nom" :"Martin","age" :30}
`_id : représente l'identifiant du document`

Consultation d'un document spécifique dans une collection

find la commande suivante permet de consulter le(s) document(s) contenant le nom *Martin* dans la base de données :
db.personnes.find({"name" :"Martin"}).

La réponse de Mongodb est retournée comme suit :
{"_id" :ObjectId("550744..."),"nom" :"Martin","age" :30}
`_id : représete l'identifiant du document`

Consultation d'un document spécifique dans une collection

find la commande suivante permet de consulter le(s) document(s) contenant le nom *Martin* et l'age 30 dans la base de données :
db.personnes.find({"name" :"Martin","age" :30}).

La réponse de Mongodb est retournée comme suit :
{"_id" :ObjectId("550744..."),"nom" :"Martin","age" :30}
`_id : représente l'identifiant du document`

Consultation suivi d'une projection d'un document spécifique dans personnes

find la commande suivante permet de consulter le(s) document(s) dont le nom est *Martin* et afficher seulement leur age :
db.users.find({"name" :"Martin"},"age" :1).

La réponse de Mongodb est retournée comme suit :
{"_id" :ObjectId("550744..."),"age" :30}
`_id : représente l'identifiant du document`

Consultation utilisant un interval: $gt,$gte,$lt,$lte

find la commande suivante permet de consulter le(s) personnes dont l'age est supérieur à 20 et afficher leur nom :
db.users.find({"age" :{$gt : 20}},"nom" :1).
La réponse de Mongodb est retournée comme suit :
{"_id" :ObjectId("550744..."),"nom" :"Martin"}
`_id : représente l'identifiant du document`

Insertion d'un deuxième document dans personnes

insert On insère un autre document dans la collection *personnes* :

db.personnes.insert({"nom" :"Dupont","age" :40})

Consultation utilisant des opérateurs logiques: $or,$and,$not

find la commande suivante permet de consulter le(s) document(s) dont l'age est supérieur à 20 et inférieur à 50. On affiche le nom :
db.personnes.find({$and :["age" :{$gt : 20},"age" :{$lte : 50}]},"nom" :1).
La réponse de Mongodb est retournée comme suit :
{"_id" :ObjectId("550745..."),"nom" :"Dupont"}
`_id : représete l'identifiant du document`

Consultation utilisant des opérateurs logiques: $or,$and,$not

find la commande suivante permet de consulter le(s) document(s) des personnes dont les noms ne commencent pas par D. On affiche leur age :

db.personnes.find({"nom" :{$not :/D/},"age" :1).

La réponse de Mongodb est retournée comme suit :

{"_id" :ObjectId("550744..."),"age" :30}

`_id : représente l'identifiant du document`

Mise à jour de documents

update la commande suivante permet de mettre à jour de(s) document(s) dans une collection.
L'exemple suivant modifie l'age de Martin à 20 (option set).

```
db.personnes.update({"nom" :"Martin"},{$set :{"age" :20}})
```

Pour vérifier le changement il suffit de lancer la commande find.

Mise à jour de documents

update L'exemple suivant supprime le champs age de Martin dans la base de données (option unset).

db.personnes.update({"nom" :"Martin"},{unset : {age :1}})

Mise à jour de documents

update L'exemple suivant ajoute le champs age à Martin dans la base de données (option push).

db.personnes.update({"nom" :"Martin"},{push : {age :1}})

Suppression de documents

remove la commande suivante permet de supprimer tous les document(s) dans la collection personnes.

```
db.personnes.remove()
```

Suppression de documents

remove la commande suivante permet de supprimer seulement les personnes dont l'age est supérieur à 20.

```
db.personnes.remove({"age" :{$gt :20}})
```

Manipulation des types complexes : Arrays

insert la commande suivante permet d'insérer un document contenant un champ de type Array dans la collection *resto*.

db.resto.insert({"menu" :"tomate","fromage","beurre"})
Nous insérons un deuxième document dans *resto* :

db.resto.insert({"menu" :"pizza","frites","beurre"})

Recherche dans un Array

insert la commande suivante permet de rechercher un menu contenant du fromage.

```
db.resto.find({"menu" :"fromage"})
```

la commande suivante permet de rechercher un menu contenant les deux ingrédients fromage et beurre.

```
db.resto.find({"menu" :{$all :["fromage","beurre"]}})
```

Document imbriqué

- Un document imbriqué est utilisé pour organiser les données dans une structure de données non plate.

insert la commande suivante permet d'insérer un document imbriqué dans la collection *auteur*. Dans cet exemple *Durand* a deux livres.
db.auteur.insert({ "ida" : "a0", "nom" : "Durand", "prenom" : "Robert", "livres" : [{ "idl" : "l1", "thème" : "policier" }, { "idl" : "l2","thème" : "SF" }]}

Recherche dans un document imbriqué

insert la commande suivante permet de rechercher les auteurs des livres du thème Policier.

```
db.auteur.find({"livres.thème" :"Policier"},{"nom" :1})
```

Insertion d'un document imbriqué

insert la commande suivante permet d'insérer un document imbriqué dans la collection *auteur*. Dans cet exemple on assigne le document imbriqué a une variable Mongodb. Ceci est possible puisque Mongodb supporte JavaScript.

x= { "ida" : a1, "nom" : "Martin", "prenom" : "Sophie", "livres" : [{ "idl" : "l1", "thème" : "policier" }, { "idl" : "l3","thème" : "Histoire" }]}

db.auteur.insert(x)

Utilisation de fonctions spécifiques

pretty Cette fonction appliquée à une requête de consultation find permet de mieux présenter le résultat de recherche.

```
db.auteur.find({"livres.thème" :"Policier"},{"nom" :1}).pretty()
```

Utilisation de fonctions spécifiques

limit Cette fonction appliquée à une requête de consultation find permet de limiter le nombre de résultats retournés.

```
db.auteur.find({"livres.thème" :"Policier"},{"nom" :1}).limit(2)
```

Utilisation de fonctions spécifiques

sort Cette fonction appliquée à une requête de consultation find permet de trier le résultat selon les valeurs d'un champs (ou plusieurs).

```
db.auteur.find({"livres.thème" :"Policier"},
{"nom" :1}).sort({"nom" :1})
```

Utilisation de fonctions spécifiques

count Cette fonction appliquée à une requête de consultation find permet de compter le nombre de documents retournés dans le résultat.
db.auteur.find({"livres.thème" :"Policier"},{"nom" :1}).count()

Utilisation des Agrégats

L'agrégation dans Mongodb permet de traiter successivement un ensemble d'opérations : le résultat retourné par la première opération est récupéré par l'opération suivante qui effectue un traitement et renvoie le résultat à l'opération suivante formant ainsi un *pipeline* : la sortie d'une opération est l'entrée de l'opération suivante. La dernière opération retourne le résultat final de l'agrégation.

Utilisation des Agrégats

Plusieurs opérations sont utilisées dans une agrégation. Nous distinguons les opérations suivantes :

1. $match
2. $project
3. $group
4. $unwind
5. $sort
6. $limit
7. $lookup

`Utilisation des Agrégats`

Opération **`$match`** :

Cette opération permet de filtrer les documents d'une collection selon un ou plusieurs critères

Exemple : dans la collection auteur je m'intéresse aux livres de thème Policier. db.auteur.agregate({"$match" :{"livres.theme :"Policier"} })

`Utilisation des Agrégats`

Opération **`$project`** :

Cette opération permet de projeter les documents sélectionnés dans une collection sur un ou plusieurs champs

Exemple : dans la collection auteur je m'intéresse aux noms des auteurs des livres de thème Policier.

db.auteur.agregate({"$match" :{"livres.theme :"Policier"} },
{"$project" :"nom" :1 })

Utilisation des Agrégats

Opération **$group** :

Cette opération permet de regrouper les documents d'une collection selon les valeurs d'un ou plusieurs champs et d'appliquer ensuite une fonction d'agrégat sur les groupes

Exemple : dans la collection auteur on cherche à connaitre le nombre de livres par thème concernant l'auteur *Martin*.

```
db.auteur.agregate({"$match" :{"nom :"Martin"} },
{"$project" :"livres.theme" :1 },
{"$group" :{"_id" :"$livres.theme","total" :{sum :1} } })
```

`Utilisation des Agrégats`

Opération **`$sort`** :

Cette opération permet de trier le résultat en ordre croissant ou décroissant selon un ou des champs du résultat

Exemple : On souhaite trier le résultat en ordre croissant selon le nombre total de livres par thème.

db.auteur.agregate({"$match" :{"nom :"Martin"} },
{"$project" :"livres.theme" :1 },
{"$group" :{"_id" :"$livres.theme","total" :{sum :1}},{"$sort" :{"total" :1
} } })

`Utilisation des Agrégats`

Opération `$limit` :

Cette opération permet de limiter le nombre de réponse dans le résultat

Exemple : On souhaite obtenir seulement les deux thèmes les plus important parmi les livres publiés par Martin..

```
db.auteur.agregate({"$match" :{"nom :"Martin"} },
{"$project" :"livres.theme" :1 },
{"$group" :{"_id" :"$livres.theme","total" :{sum :1}},{"$sort" :{"total" :1
}},{"$limit" :2 } })
```

`Utilisation des Agrégats`

Opération **`$unwind`** :

Cette opération permet de remplacer un document contenant un champs de type **Array** par une suite de documents contenant chacun un seul élément du tableau.

Exemple : On souhaite obtenir les thèmes publiés par Martin.

db.auteur.agregate(["$match" :{"nom :"Martin"},
{"$unwind" :"$livres.theme"}])

Avec l'opération `unwind` la sortie comprendra un document pour chacun des thèmes de l'auteur Martin.

`Utilisation des Agrégats`

Opération `$lookup` :

Cette commande permet de faire la jointure entre deux collections.

Exemple : On souhaite faire la jointure entre deux collections auteur (idlivre) et livre (numlivre)

db.auteur.aggregate([{"$lookup" : { "localField" : "idlivre", "from" : "livre", "foreignField" : "numlivre", "as" : "titre" } }, {"$out" : "resultat"}])

Le résultat de cette requête se trouve dans la collection résultat contenant tous les auteurs avec les titres de leurs livres.

`D'autres commandes` Voir la documentation disponible sur le site MongoDB à l'adresse suivante :

https ://www.mongodb.com/docs/